Aritmetica en prolog

El predicado “is”

En Prolog no pasa como en Haskell que tengo funciones que me devuelven cosas, sino que tengo predicados que relacionan individuos ¿Entonces qué sucede cuando quiero hacer cuentas? Si probamos esto en el intérprete de Prolog, que está pensado para evaluar predicados, vamos a ver que falla:

```
?- 2+2.
ERROR: Undefined procedure: (+)/2 (DWIM could not correct goal)
```

Eso no es porque no sea posible sumar números, sino que el + no es un predicado. De hecho, si probamos esto otro, sí funciona:

```
?- 2+2 > 1.
Yes
```

Eso demuestra que la suma anda, y que el predicado (>)/2 se encargó de reducir esa expresión y compararla con el 1, lo cual dio un resultado booleano como hubiéramos esperado.

¿Y como me puedo saber cuál es el resultado de una cuenta? ¿O si una cuenta da un determinado resultado?

Bueno, para esos casos, existe el predicado is, que es un predicado de aridad 2 que se puede escribir de forma infija. Al ser un predicado, sabemos que podemos hacer consultas individuales como la siguiente, para saber si es cierto que se verifica que una cuenta da un determinado resultado:

```
?- 5 is 2 + 3.
Yes
```

Pero **ojo**, la cuenta sólo puede ir a la derecha:

```
?- 4 + 1 is 2 + 3.
No.
```

A la derecha del is se escribe una operación aritmética. A la izquierda del is se escribe el resultado de esa operación aritmética.

Luego podemos ver qué tan inversible es para determinar qué otros usos se le puede dar. El is **sólo es inversible por el primer parámetro**

Esto funciona:

```
?- X is 6 / 2.
X = 3
```

Pero esto no:

```
?- 3 is X / 2.
ERROR: is/2: Arguments are not sufficiently instantiated
```

Errores comunes con el is

Acá vienen una serie de warnings que deben tener MUY en cuenta:

Error: usar = en vez de is

Usar = para resolver operaciones aritméticas no es correcto. ¿Por qué no se puede usar = para aritmética? Veamos un ejemplo:

```
?- 3+5 = 2+6.
No
?- 3+5 = 5+3.
No
?- 3+5 = 8.
No
```

El muy simpático sólo nos va a decir true cuando las dos expresiones de ambos lados del igual sean idénticas, no va a intentar resolver la igualdad:

```
?- 3+5 = 3+5.
Yes
```

Muy… “útil”, ¿no? :P

En general en la materia no vamos a usar el = ya que preferimos el uso de pattern matching y consultas individuales cuando eso nos sirva para resolver el problema que tenemos.

Error: tratar de acumular

Como ya saben, en lógico no hay asignación sino unificación; una vez que las variables se ligan, permanecen ligadas hasta que termine la consulta, por ende no hay que pensar al is como un mecanismo para asignar. Es decir, no vale preguntar algo como:

```
?- edad(pepe,E), E is E + 1.
No
```

No existe ningún número E que sea igual a E + 1. Supongamos que la edad de pepe era 15, el motivo por el cual es falso es porque 15 no es 16. Básicamente lo que estaría pasando es esto:

```
?- 15 is 15 + 1.
No
```

Error: Usar is para verificar igualdad de valores

El is es **sólo para operaciones aritméticas**. Si bien funciona, lo siguiente es un error conceptual:

```
edad(juan,Edad) :-
  Edad is 10.
```

La manera correcta de hacerlo es aprovechando que al hacer pattern matching, prolog *verifica que el valor sea ese*, sin usar el igual ó el is:

```
edad(juan,10).
```

¡Y listo!

Ver Sobre el uso del igual en Prolog (sobre-el-uso-del-igual-----en-prolog.html) para más información.

Error: Usar is para igualar variables

Esto es el mismo caso que el error anterior. El is es **sólo para operaciones aritméticas**. Si bien funciona, lo siguiente es un error conceptual:

```
mismaEdad(PersonaA,PersonaB):-
  edad(PersonaA,EdadA),
  edad(PersonaB,EdadB),
  EdadA is EdadB
```

Lo mismo sucede aquí:

```
mismaEdad(PersonaA,PersonaB):-
  edad(PersonaA,EdadA),
  edad(PersonaB,EdadB),
  EdadA = EdadB
```

La manera correcta de hacerlo es:

```
mismaEdad(PersonaA,PersonaB):-
  edad(PersonaA,Edad),
  edad(PersonaB,Edad).
```

Porque si dos individuos deben ser el mismo, entonces alcanza con escribirlos con la misma variable, no como otra condición. De esta forma estamos haciendo una **consulta existencial** respecto a la edad de la PersonaA y luego una **consulta individual** para validar si es cierto que la edad de PersonaB es esa que ya conocemos. Vemos que de ésta manera mejoramos la **declaratividad** (leo qué es lo que quiero, y hay menos detalles algorítmicos: si la edad es la misma lo escribo igual y listo). El motor de Prolog se encarga del resto.

Ver Sobre el uso del igual en Prolog (sobre-el-uso-del-igual-----en-prolog.html) para más información.

Error: Usar is para resolver ecuaciones

Como dijimos arriba, el predicado is/2 **no es completamente inversible**, es inversible solo para el primer argumento, lo que va a la izquierda. O sea, no vale preguntar:

```
?- 3 is X + 1.
ERROR: is/2: Arguments are not sufficiently instantiated
```

Porque el motor de Prolog no sabe resolver ecuaciones. En éste caso, el error dirá algo como “Arguments are not sufficiently instantiated”, lo cual es otra manera de decir “No soy inversible a derecha, sólo me podés poner algo sin unificar a izquierda”